

Formal Languages and Compilers

Prof. Breveglieri, Morzenti, Agosta

Written exam: laboratory question

25/01/2024

Time: 60 minutes. Textbooks and notes can be used. Pencil writing is allowed.

Important: Write your name on any additional sheet.

SURNAME (Cognome):

NAME (Nome):

Matricola:or Person Code:

Instructor: ☐ Prof. Breviglieri ☐ Prof. Morzenti ☐ Prof. Agosta

The laboratory question must be answered taking into account the implementation of the Acse compiler given with the exam text.

Modify the specification of the lexical analyser (flex input) and the syntactic analyser (bison input) and any other source file required to extend the Lance language with the ability to evaluate **iterated expressions**.

This operation is available through a new statement called **repeat_exp**, having the following syntax:

```
repeat_exp (⟨var.⟩ = ⟨exp. 1⟩, ⟨exp. 2⟩, ⟨exp. 3⟩);
```

The `repeat_exp` statement implements a simple loop which, at every iteration, re-computes $\langle exp. 3 \rangle$ and assigns it to $\langle var. \rangle$. $\langle var. \rangle$ is initialized to the value of $\langle exp. 1 \rangle$ before the loop. The number of iterations is controlled by the value of $\langle exp. 2 \rangle$.

If $\langle exp. 2 \rangle$ is zero or negative, the loop should never iterate, leaving $\langle var. \rangle$ to its initial value, i.e. $\langle exp. 1 \rangle$. Even though this is allowed by the syntax, assume that $\langle exp. 2 \rangle$ does not use $\langle var. \rangle$ in its definition. As a result you can assume that $\langle exp. 2 \rangle$ doesn't change during the execution of the loop.

The following code snippet exemplifies the operation of the `repeat_exp` statement. The first `repeat_exp` statement initializes a to zero, then performs the assignment $a \leftarrow a + 1$ ten times — as specified by the $\langle exp. 2 \rangle$ argument — leaving $a = 10$ at the end. The second `repeat_exp` initializes b to $a \times 3 = 30$, but then $\langle exp. 2 \rangle$ evaluates to -5 , which is a negative value. As a result, b is left equal to 30 and is not changed again. The third occurrence of `repeat_exp` implements a variant of the well-known recurrent formula $x_{i+1} = \frac{1}{2} \left(x_i + \frac{N}{x_i} \right)$ to compute $\sqrt{4096576}$.

```
int a, b;

repeat_exp(a=0, 10, a+1);
// a == 10

repeat_exp(b=a*3, a-15, b-1);
// b == 30

repeat_exp(a=1000, 3, (a+4096576/a)/2);
// a == 2024
```

1. Define the tokens (and the related declarations in **Acse.lex** and **Acse.y**). (1 points)
2. Define the syntactic rules or the modifications required to the existing ones. (2 points)
3. Define the semantic actions needed to implement the required functionality. (22 points)

4. Given the following Lance code snippet:

```
int a[6]; c=13+2;
```

write down the syntactic tree generated during the parsing with the Bison grammar described in Acse.y *starting from the axiom*. (5 points)

5. (**Bonus**) Briefly explain in plain English words how to modify the solution given to the questions about the `repeat_exp` statement in order to stop the loop earlier if the value of the variable stays the same for at least two consecutive iterations.